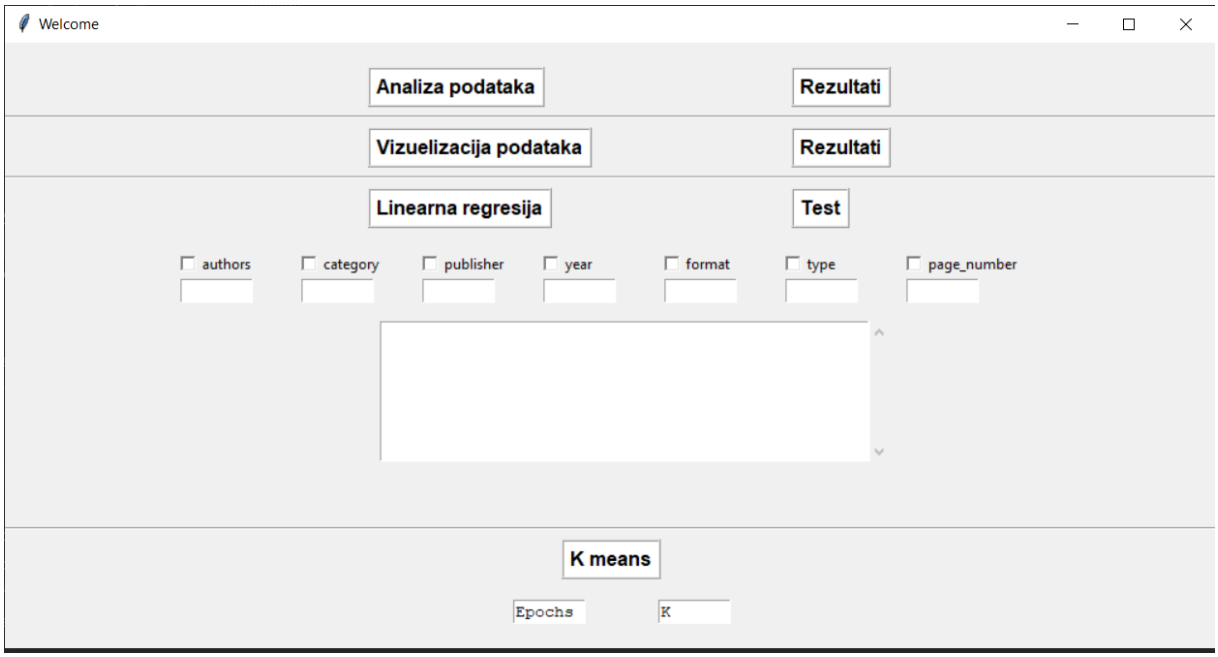
IZVEŠTAJ

Pronalaženje skrivenog znanja

Stefan Kangrga-Mikulić 2023/3242

1. Višestruka linearna regresija

Cela aplikacija se pokreće pokretanjem programa implementirane u GUI.py fajlu.



Pre pokretanja obučavanje modela linearne regresije potrebno je odabrati ulazne atribute (autor, žanr/kategorija, izdavač, godina izdanja, broj strana, informaciju o povezu i formatu). Za atribut format racuna se površina knjige čija se vrednost nalazi u koloni „area“ u MySQL bazi. Nakon pokretanja obučavanja, dohvataju se podaci iz baze po onim atributima/odlikama koji su odabrani.

```
connection = mysql.connector.connect(host='localhost',
        user='root',
        passwd='0505998710353',
        database='knjige')
cursor = connection.cursor()
column_names = ''
for selected_item in selected_parameters_list:
    if selected_item == 'format':
        column_names = column_names + 'area' + ','
    else:
        column_names = column_names + selected_item + ','

column_names += 'price'
sql_script = "SELECT " + column_names + " FROM knjige"

my_data = pd.read_sql(sql_script, con=connection)
```

Učitavanje podataka iz baza

Za kolone čije vrednosti nisu numeričke radi se One-hot encoding kako bi se svakoj kategoriji pridružila nova kolona i vrednost 0 ili 1 i na taj način se omogućava da svaka kategorija za svaki atribut ima jednaku važnost prilikom obučavanja(kad bi se za svaku kategoriju davale vrednosti redom 0,1,2... ispalo bi da je jedna kategorija „važnija“ za obučavanje od neke druge kategorije sa manjom pridruženom numeričkom vrednošću). Nakon zamene kategoričkih vrednosti sa numeričkim radi se normalizacija kako bi se ubrzao gradijentni spust do minimuma funkcije greške.

```
my_data = pd.read_sql(sql_script, con=connection)
print('======================PRE PRIMENE ONE HOT ENCODING=======================')
print(my_data)

for selected_item in selected_parameters_list:
    if selected_item != 'year' and selected_item != 'page_number' and selected_item != 'format':
        my_data = pd.get_dummies(my_data, columns=[selected_item], dtype='int')

# my_data = pd.get_dummies(my_data, columns=['type'], dtype='int')
# my_data = pd.get_dummies(my_data, columns=['category'], dtype='int')

print('======================POSLE PRIMENE ONE HOT ENCODING=======================')
print('Nove kolone: ', my_data.columns)   #my_data.head()
print('Broj kolona: ', my_data.shape[1])
```

One Hot encoding

```
#NORMALIZACIJA
colums = my_data.columns[0]
max = [my_data[c].max() for c in my_data.columns]
min = [my_data[c].min() for c in my_data.columns]
print('Maksimalne vrednosti po kolonama: ', max)
print('Minimalne vrednosti po kolonama: ', min)
i = 0
for c in my_data.columns:
    while(i<len(my_data.columns)):
        my_data[c]=(my_data[c]-min[i])/(max[i]-min[i])
        i=i+1
        break
```

Normalizacija

```
# Select ratio
ratio = 0.75

total_rows = df.shape[0]
train_size = int(total_rows * ratio)

# Split data into test and train
train = df[0:train_size]
test = df[train_size:]
train_without_price = df[0:train_size].drop(columns = ['price'])
test_without_price = df[train_size:].drop(columns = ['price'])

print('Skup za obucavanje sa izlazom: ', train)
print('Velicina skupa za obucavanje: ', train.shape)
print('Skup za testiranje: ', test)
print('Velicina skupa za testiranje sa izlazom: ', test.shape)


X_train = np.matrix(train_without_price)
X_train = np.concatenate( arrays: (np.ones(X_train.shape[0])[:, np.newaxis], X_train), axis=1)
Y_train = np.asarray(train["price"].values.tolist())
Y_train = Y_train.reshape(len(Y_train), 1)

X_test = np.matrix(test_without_price)
X_test = np.concatenate( arrays: (np.ones(X_test.shape[0])[:, np.newaxis], X_test), axis=1)

Y_test = np.asarray(test["price"].values.tolist())
Y_test = Y_test.reshape(len(Y_test), 1)
```

Podela podataka na ulaz i izlaz i na skup za obučavanje i testiranje

```
#Randomizacija pocetnih parametara modela
coef = np.random.randint(low=9, high=15, size=(1, X_train.shape[1]))
coef = 0.01 * coef
print('Pocetni parametri modela: ', coef)
print('coef length: ', coef.shape)

coef_finally = multilinear_regression(text_widget, coef, X_train, Y_train,  lr: 1e-2)
print('Konacni parametri modela: ', coef_finally )
print('Velicina matrice: ', coef_finally.shape)
```

Inicijalizacija parametara modela

Prilikom podele podataka na ulazne i izlazne parametre kao i na podatke za obučavanje i kasnije testiranje, u ulaznim podacima se na početak dodaje kolona sa vrednostima 1 kako bi se množilo sa težinskim parametrom w_0. Obučavanje modela se pokreće pozivanjem funkcije multilinear_regression koji u while petlji računa gresku modela na osnovu množenja matrice $X_{MxN} \cdot W_{Nx1}$ i sumiranjem vrednosti, nakon čega se radi jedan korak grupnog gradijentnog spusta, azuiraju listu težinskih parametara i koraci se ponavljaju dok promena funkcija greške ne bude ispod 0,00001. Koristi se i L2 regularijacija za slučaj da „greben“ minimalnih vrednosti zameni jedinstvenim minimumom(jer postoji mogućnost da ima visoka korelisanost između odlika).

```python
def multilinear_regression(text_widget, coef, x, y, lr, b1=0.9, b2=0.999, epsilon=1e-5):
    prev_error = 0

    while True:
        error = mse(coef, x, y)
        text_widget.insert("1.0", error)
        text_widget.insert("1.0", '\n')
        print('Loss:', error)
        if abs(error - prev_error) <= epsilon:
            break
        prev_error = error
        grad = gradients(coef, x, y)

        delta = lr * grad
        regularisation_part1 = coef * lr * (0.5/x.shape[0])
        regularisation = coef - regularisation_part1
        coef = np.subtract(regularisation, delta.transpose())
    print('Kraj')
    return coef
```

$$w_r\left(1-\alpha\frac{\lambda}{m}\right)-\alpha\frac{1}{m}\left(\sum_{i=1}^{m}\left(h\left(x^{(i)}\right)-y^{(i)}\right)x_r^{(i)}\right)$$

```python
def mse(coef, x, y):
    # # print('MSE dot5: ', np.mean(np.power(((np.dot(x, coef.transpose())) - y), 2)) / 2)
    # # print('dot5 length: ', , 2)) / 2).shape)
    # print('MSE dot5: ', ((np um()/my_data.shape[0])
    # print('dot5 length: ', ( 2).sum()/my_data.shape[0]).shape)

    return np.sum(((np.power(((np.dot(x, coef.transpose())) - y), 2)) / 2))/x.shape[0]
```

$$J(w)=\frac{1}{2m}\sum_{i=1}^{m}\left(w_0+w_1x_1^{(i)}+w_2x_2^{(i)}+\cdots+w_nx_n^{(i)}-y^{(i)}\right)$$

Računanje funkcije greške

```python
def gradients(coef, x, y):
    # print('dot4 length: ', ((np.dot(x.transpose(), (np.dot(x, coef.transpose()) - y)))/my_data.shape[1]).shape)
    # return (np.dot(x.transpose(), (np.dot(x, coef.transpose()) - y)))/my_data.shape[1]

    result_delta = np.array([])
    new_matrix = np.dot(x, coef.transpose()) - y
    index1 = x.shape[1]
    for i in range(index1):
        sum = 0.0
        index2 = x.shape[0]
        # print('i:::', i)
        for j in range(index2):
            sum += new_matrix[j, 0] * x[j, i]
            # print('sum:::', new_matrix[
        # new_delta = ((np.dot(x, coef.tr
        # result = np.array([new_delta])
        sum /= x.shape[0]
        result = np.array([sum])
        result_delta = np.append(result_delta, result, axis=0)

    return result_delta.transpose()
```

$$w_r\left(1-\alpha\frac{\lambda}{m}\right)-\alpha\frac{1}{m}\left(\sum_{i=1}^{m}\left(h\left(x^{(i)}\right)-y^{(i)}\right)x_r^{(i)}\right)$$

Primena grupnog gradijentnog spusta

Prikaz rezultata obučavanja jeste R2 score nad podacima za obučavanje i podacima za testiranje odvojeno.

```python
def r2_score(Ypred, Y):

    print('sum/len', sum(Y)/len(Y))
    print('mean', np.average(Y))

    mean = np.average(Y)
    sum1 = np.sum((np.power(np.subtract(Ypred, mean), 2)))  # the sum of square(due to regression)
    # sum3 = np.sum((np.power(np.subtract(Y, Ypred), 2)))  # the sum of square(due to error)
    sum2 = np.sum((np.power(np.subtract(Y, mean), 2)))      # the sum of square(total)

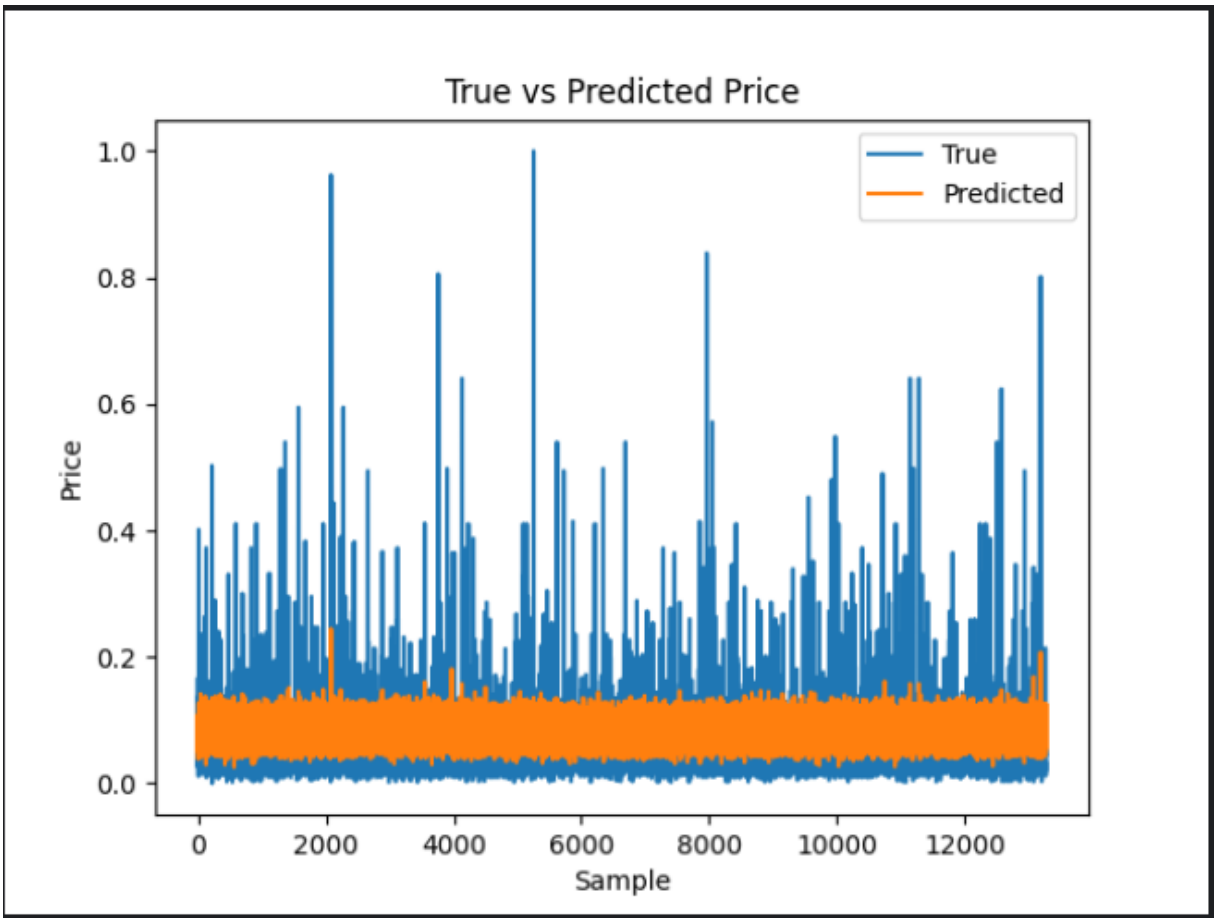
    # print('1: ', np.subtract(Ypred, mean))
    # print('2: ', np.power(np.subtract(Ypred, mean), 2))

    print('sum1: ', np.sum((np.power(np.subtract(Ypred, mean), 2))))
    print('sum2: ', sum2)
    r2 = sum1/sum2
```

$$R^2 = \frac{\sum(y_{pred} - \bar{y})^2}{\sum(y - \bar{y})^2}$$

R2 score

```
Velicina Y_pred:  (13266, 1)
sum/len [0.0534847]
mean 0.05348469681594826
sum1:  15.617827851738868
sum2:  34.470583182894174
R2 score 0.45307698360871146
Y_pred:  [[0.05457688]
 [0.11769351]
 [0.05290192]
 ...
 [0.06246047]
 [0.05342854]
 [0.11115161]]
Velicina Y_pred:  (4423, 1)
sum/len [0.05402815]
mean 0.054028151656751136
sum1:  5.075408568486583
sum2:  11.502683715690262
R2 score nad testnim podacima 0.4412369055721718
```



2. K-srednjih vrednosti (k-Means)

Cela aplikacija se pokreće pokretanjem programa implementirane u GUI.py fajlu. Pre pokretanja klasterizacije potrebno je odabrati ulazne atribute (autor, žanr/kategorija, izdavač, godina izdanja, broj strana, informaciju o povezu i formatu). Za atribut format racuna se površina knjige čija se vrednost nalazi u koloni „area“ u MySQL bazi. Nakon pokretanja klasterizacije, dohvataju se podaci iz baze po onim atributima/odlikama koji su odabrani.

```
connection = mysql.connector.connect(host='localhost',
                                     user='root',
                                     passwd='0505998710353',
                                     database='knjige')
cursor = connection.cursor()
column_names = ''
for selected_item in selected_parameters_list:
    if selected_item == 'format':
        column_names = column_names + 'area' + ','
    else:
        column_names = column_names + selected_item + ','

column_names += 'price'
sql_script = "SELECT " + column_names + " FROM knjige"

my_data = pd.read_sql(sql_script, con=connection)
```

Učitavanje podataka iz baza

Za kolone čije vrednosti nisu numeričke radi se label encoding kako bi se svakoj kategoriji pridružila nova kolona i vrednost 0...n. Svi podaci se kastuju iz objekta koji naliči na niz u matricu numeričkih vrednosti.

Pozivom funkcije train_k_means_clustering pokreće se algoritam gde se prvo nasumično nalaze koordinate centroida(onoliko koliko je broj klastera definisano). Pre prolaska kroz petlju definisanom vrednošću epochs, svakom podatku se određuje kom klasteru pripada i vrednosti klastera (0,1…,k) se dodaju na novu kolonu u matrici za svaki podatak(funkcija point_clustering).

```python
def train_k_means_clustering(data, k=2, epochs=5):
    dims = len(data[0])
    print('data[0]:', data[0])
    centers = random_centers(dims, k)

    clustered_data = point_clustering(data, centers, dims, first_cluster=True)

    for i in range(epochs):
        centers = mean_center(clustered_data, centers, dims)
        clustered_data = point_clustering(data, centers, dims, first_cluster=False)

    return centers
```

```python
def random_centers(dim,k):
    centers = []
    for i in range(k):
        center = []
        for d in range(dim):
            rand = random.randint( a: 0, b: 100)
            center.append(rand)
        centers.append(center)
    return centers
```

Randomizacija početnih centroida klastera

```python
def point_clustering(data, centers, dims, first_cluster=False):
    for point in data:
        nearest_center = 0
        nearest_center_dist = None
        for i in range(0, len(centers)):
            euclidean_dist = 0
            for d in range(0, dims):
                dist = abs(point[d] - centers[i][d])
                euclidean_dist += dist
            euclidean_dist = np.sqrt(euclidean_dist)
            if nearest_center_dist == None:
                nearest_center_dist = euclidean_dist
                nearest_center = i
            elif nearest_center_dist > euclidean_dist:
                nearest_center_dist = euclidean_dist
                nearest_center = i
        if first_cluster:
            point.append(nearest_center)
        else:
            point[-1] = nearest_center
    return data
```

Određivanje pripradnosti klastera za svaki podatak

Kroz for petlju, u funkciji mean_center se odredjuju nove koordinate centroida za svaki klaster na osnovu srednje vrednosti koordinata svih podataka koji pripadaju datom klasteru(zadnja vrednost za svaki podatak predstavlja pripradnost klastera).

```python
def mean_center(data, centers, dims):
    print('centers:', centers, 'dims:', dims)
    new_centers = []
    for i in range(len(centers)):
        new_center = []
        n_of_points = 0
        total_of_points = []
        for point in data:
            if point[-1] == i:
                n_of_points += 1
                for dim in range(0, dims):
                    if dim < len(total_of_points):
                        total_of_points[dim] += point[dim]
                    else:
                        total_of_points.append(point[dim])
        if len(total_of_points) != 0:
            for dim in range(0, dims):
                # print(total_of_points, dim)
                new_center.append(total_of_points[dim] / n_of_points)
            new_centers.append(new_center)
        else:
            new_centers.append(centers[i])

    return new_centers
```

Određivanje novih centroida na osnovu podataka koji pripadaju datim klasterima

Nakon toga se iznova na osnovu novih centroida odredjuju pripadnost podataka klasterima(funkcija point_clustering).

